Project Prompt – Enterprise Professional Services Automation (PSA) on SAP BTP

Paste this prompt into an AI coding assistant to rebuild the project from scratch. It contains the full problem statement, the functional requirements (from the FRD and PPT in `docs/requirements`), the design decisions taken while building it, and the pitfalls that were found and fixed along the way.

---

1. Your task

Build a complete, working **Enterprise Professional Services Automation (PSA)** application on **SAP BTP** using:

- **SAP CAP (Node.js, ESM)**: OData V4 services and business logic
- **SAP HANA Cloud**: persistence (HDI container)
- **SAP Fiori Elements V4 / SAPUI5**: user interface, started from a SAP Fiori launchpad (sandbox)
- **XSUAA + SAP Approuter**: login and roles
- **MTA deployment to Cloud Foundry**: public URL, so users can log in from anywhere

Hard constraints:

1. **Free tier / trial only.** Use nothing paid: HANA Cloud plan `hana-free`, HDI `hdi-shared`, XSUAA `application`, CF runtime ≤ 1 GB in total.
2. **Use only SAP technology.** Use SAP HANA, CAP, Fiori/UI5, XSUAA and the Approuter, and nothing else. SQLite is allowed only as a dev-dependency so that `npm test` runs without HANA.
3. **Keep it simple and explainable.** A student team must be able to explain every file. Use no extra frameworks and no unnecessary services (no Work Zone, Destination, DIM, Alert Notification, Event Mesh or CI/CD service). Simplify the FRD wherever that does not hurt the problem statement.
4. **The entities from the problem statement are hard requirements.** The PPT may change.
5. It must run in SAP Business Application Studio with exactly these commands:

```
cds build --production
cds deploy --to hana
cds watch --profile hybrid
```

6. It must deploy with `mbt build` and `cf deploy mta_archives/psa-application_1.0.0.mtar`.
7. Do not commit personal e-mail addresses. Seed data uses `@psa-demo.com` addresses.

---

2. Problem statement (requirements.txt)

> Professional services organizations such as IT consulting firms, engineering agencies and marketing groups depend on tracking time against explicit project scopes to stay profitable. Today, project data, time logs and invoicing live in disconnected tools. Team members log hours in third-party trackers, finance teams reconcile spreadsheets by hand for monthly invoicing, and project managers see budget consumption only after month end.
>
> This project builds a unified Professional Services Automation (PSA) application on SAP BTP using the Cloud Application Programming (CAP) Model (Node.js) with a SAP Fiori (Fiori Elements) frontend, deployed as an MTA to Cloud Foundry. The solution models the real hierarchy of work: **a Client has Projects; each Project has one Budget and is broken into Milestones and Tasks; Employees log hours against Tasks through Timesheets; approved hours drive real-time budget consumption and automated Invoices.**

Business problems and how they must be solved

Problem	Required solution
Scope creep: a task burns hours unseen until the budget is breached	Track task actual vs. estimated hours (burn %), with alerts at 75 %, 90 % and 100 % per task and per project
Disconnected systems: hours in third-party trackers, spreadsheets for invoicing	Generate invoices directly from approved, billable timesheet hours (Fixed, T&M, Retainer) and link the billed hours so they are never billed twice
No real-time tracking: budget visible only at month end	When a timesheet is approved, hours roll up immediately: Entry → Task → Milestone → Project → Budget

Objectives (FRD §2.2)

- Track time at task level and roll it up to milestone, project and budget in near real time.
- Alert managers automatically when a task or project crosses consumption thresholds.
- Generate invoices from approved timesheet data, with no spreadsheet reconciliation.
- Support three billing models: **Fixed**, **Time & Materials (T&M)** and **Retainer**.
- Give every role a single source of truth with role-based visibility.

Success KPIs (FRD §2.3). Show these on the dashboard where they can be measured.

- Timesheet submission by weekly deadline ≥ 95 %
- Budget visibility latency (timesheet → budget dashboard) < 1 minute
- Projects with a scope-creep alert raised before 100 % budget ≥ 90 %
- Invoice preparation time reduced ≥ 60 %
- Invoice errors requiring credit notes < 2 %

---

3. Functional requirements (FRD §8) and how to implement them

ID	Requirement	Acceptance criteria	Implementation in this project
FR-001	Admin maintains Clients, Departments, Employees, Managers	Unique e-mail; inactive employees cannot log time	Clients (industry, status) and Employees (join date) apps (Admin). Manager = Employee with `isManager = true`. Department is left out (simplification). Employee e-mail is stored lower-case. An inactive employee cannot create a timesheet.
FR-002	Manager creates a project for a client with exactly one budget	Unique `project_code`; `budget > 0`; `end ≥ start`	Projects app with a draft. `BUDGET` is a `Composition of one`. It is created empty on *Create*, and the server validates code uniqueness, budget > 0 and the date order.
FR-003	Manager defines milestones and tasks with estimated hours	Dates inside the parent period; `estimated_hours > 0`	Milestones are a composition of the project, tasks are a composition of the milestone, edited in the project draft (sub-object page). Milestone dates must lie inside the project period and task dates inside the milestone period; validated on save. Tasks have priority, dates and status *To Do / In Progress / Done / Blocked*; the milestone status (*Not Started / In Progress / Completed / Delayed*) is set by the roll-up.
FR-004	Employee logs daily hours per task (billable or not), saves a draft and submits the week	Daily total ≤ 24 h; submit locks the entries	My Timesheets app. Header = one employee + one week (Monday–Sunday), items = one day + one task. The `submitTimesheet` action locks it (Edit/Delete hidden, server rejects changes) and stamps `SUBMITTED_DATE`. Total and billable hours are kept on the header.
FR-005	Import hours from third-party trackers (CSV/API)	Unmapped rows reported	Bound action `importHours(rows)` on a Draft/Rejected timesheet: paste CSV lines `date;task;hours;billable;notes` (task = name, or `PROJECT_CODE / name`). Valid rows are added, duplicates (same task + day) ignored, unmapped or invalid rows reported with their line number.

ID	Requirement	Acceptance criteria	Implementation in this project
FR-006	Manager approves or rejects a submitted timesheet; rejection needs a reason	Status and approver recorded	Approve Timesheets app. `approveTimesheet` stores `APPROVED_BY` and `APPROVED_DATE`. `rejectTimesheet(reason)` requires a reason. Nobody can approve their own timesheet. A rejected timesheet can be corrected and submitted again.
FR-007	Budget consumption updates dynamically	Dashboard reflects the change within 60 s	The roll-up runs synchronously in the approve action, so the change is visible instantly.
FR-008	Task-level burn tracking (actual vs. estimate)	Warning colour at 80 %, error at 100 %	`TaskSrv.CRITICALITY` (≥ 100 → red, ≥ 80 → orange, else green), progress bars in the UI.
FR-009	Automatic alerts at thresholds 75 / 90 / 100 % for task and project	One alert per threshold crossing	An `Alerts` entity, written once per (project or task, threshold), shown on the project page. The thresholds are maintained in the **Settings** app (default `75,90,100`). E-mail is out of scope.
FR-010	Finance generates invoices from approved billable hours for Fixed, T&M, Retainer	Totals reconcile to entries; tax by configuration	The `generateInvoice(projectID, periodFrom, periodTo)` action. Tax % and payment days come from the **Settings** app (defaults 18 % and 30 days).
FR-011	Invoice lifecycle; invoiced timesheets are locked	Valid transitions only; no double billing	Draft → Issued → Paid via `issueInvoice` / `markInvoicePaid`. Billed entries get `INVOICE` set, and a timesheet becomes *Invoiced* when all its billable hours are billed. `markInvoicePaid(paymentDate)` stores `PAYMENT_DATE`. `cancelInvoice` (Draft/Issued → Cancelled) unlinks the hours so they can be billed again. *Overdue* is shown when an issued invoice is past its due date (`PAYMENT_STATUS`). PDF/DIM is out of scope.
FR-012	Role-based access with row-level restrictions, audit log	Employee sees only own time	`@requires` / `@restrict` with `where: 'EMPLOYEE.email = $user'`. All entities use the `managed` aspect; an `AuditLog` entity records every saved create/update/delete (changed fields) and every status action (user, time, action, object, details), shown in the **Audit Log** app.

Business rules (FRD §6.4, §7.2), all enforced on the server

Timesheet entries. Check only the FRD rules; do not add extra restrictions:

- A task is required, and the task must exist and not be *Done*.
- Hours are between 0.25 and 24.
- `WORK_DATE` lies inside the timesheet week.
- No duplicate entry for the same task and day.
- The daily total is ≤ 24 h.
- An employee may book on **any** open task. Do not restrict to assigned tasks.

Timesheet header

- The week start is moved to the Monday of its week automatically.
- `WEEK_END` = Monday + 6.
- There is one timesheet per employee per week. If a second one is created, show "You already have a timesheet for the week of … – please open it from the list".
- Only *Draft* or *Rejected* timesheets can be edited, deleted or submitted.

Defaults (smooth UX)

- A new timesheet gets the current employee and the current week's Monday.
- A new entry gets date = week start, 8 h and billable = true.
- A new project gets billing type T&M and an empty budget.

Roll-up (`recalculateProject`), triggered on approve, on project save and by the action *Recalculate Budget*. It counts hours of timesheets with status *Approved* or *Invoiced*:

- `Task.ACTUAL_HOURS` = sum of hours; `BURN_PERCENT` = actual / estimated × 100.
- `Milestone.COMPLETION_PERCENT` = % of its tasks in status *Done*.
- `Project.CONSUMED_HOURS` = sum of task actual hours.
- `Budget.CONSUMED_AMOUNT` = billable hours × `BILLING_RATE`; `REMAINING` = budget − consumed; `CONSUMED_PERCENT`.
- `BUDGET_STATUS`: > 100 % = *Breached*, ≥ 80 % = *At Risk*, otherwise *Within Budget*.
- Alerts are raised for each threshold in [75, 90, 100] that is crossed and not yet raised.
- Run the roll-up once at server start (`cds.once('served')`) so that the seed data has calculated values.

Invoicing, using approved, billable, not yet invoiced entries whose `WORK_DATE` is inside the period:

Billing type	Net amount
T&M	hours × billing rate
Fixed	hours × rate, but never more than budget − already billed
Retainer	flat `RETAINER_FEE` per period; reject if the period overlaps an existing invoice

- Tax = 18 %, gross = net + tax, due date = invoice date + 30 days.
- Number format `INV-YYYY-0001`.
- Reject with a clear message if there are no hours (T&M/Fixed) or the contract is already fully billed.

---

4. Users and roles

The FRD defines Employee, Project Manager, Finance, Department Head and Administrator. **Simplify this to two XSUAA roles:**

Role (scope)	Role collection	Can do
Admin	`PSA_Admin`	everything: clients, employees, projects (budget, milestones, tasks), approve or reject timesheets, invoices, alerts, Manager Home dashboard, also own timesheets
User	`PSA_User`	only own timesheets: create, edit, submit. Reads tasks to pick them. Sees nothing else.

- The login e-mail (`$user`) links a user to an `Employees` record (case-insensitive).
- If a logged-in user has no employee record, show the message "your account is not linked with an employee".
- **Hide what a user may not see.** Do not show a *Forbidden* error. The launchpad offers only the apps of the user's role, and the server returns no foreign data.
- The launchpad header shows the real user name, and *Sign Out* calls `/do/logout` of the approuter.
- Provide a `whoAmI()` function (user, name, e-mail, origin, isAdmin, isUser, scopes, linked employee) for troubleshooting.
- Roles are assigned manually in the BTP cockpit (no auto-assignment), using the **Default identity provider** (origin `sap.default`).

---

5. Data model

Follow the skeleton of the sample project `SantoshSIJBS/SEP-22-2026-SKILLS-CAPJS`:

- Keys are `key NODE_KEY : cmn.Guid` and fields are UPPERCASE.
- An item points to its header with `PARENT`.
- `db/schema.cds` holds both contexts `master` and `transaction`; reusable types and aspects live in `db/common.cds`; read-only views live in `db/CDSView.cds` (`context CDSViews { define view … }`).
- Every field has a label `@(title: '{i18n>KEY}')` from `db/i18n/i18n.properties` (+ `i18n_de`, `i18n_hi`).
- CSV files are named `psa.db-master.<Entity>.csv` / `psa.db-transaction.<Entity>.csv`.
- Service projections are named `<Entity>Srv`.

`db/common.cds`, namespace `psa.common`:

- Types: `Guid : UUID`, `PhoneNumber`, `Email`, `String32/64/255`, `AmountT : Decimal(15,2)`, `Hours : Decimal(10,2)`, `Percent : Decimal(7,2)`.
- Enums:
  - `BillingType` (Fixed, T&M, Retainer)
  - `BudgetStatus` (Within Budget, At Risk, Breached)
  - `TaskStatus` (To Do, In Progress, Done)
  - `TimesheetStatus` (Draft, Submitted, Approved, Rejected, Invoiced)
  - `InvoiceStatus` (Draft, Issued, Paid)
- Aspects:
  - `Amount` (GROSS_AMOUNT, NET_AMOUNT, TAX_AMOUNT, CURRENCY : Currency), with i18n titles
  - `Address` (STREET, POSTAL_CODE, CITY, COUNTRY, BUILDING), with i18n titles

`db/schema.cds`, namespace `psa.db`, part 1: `context master`.

- **Clients** : `common.Address`, `managed` – NODE_KEY, CLIENT_NAME, CONTACT_PERSON, EMAIL, PHONE, INDUSTRY, STATUS (Active/Inactive/Prospect)
- **Employees** : `cuid`, `managed` – nameFirst, nameLast, email, phoneNumber, jobTitle, joinDate, Currency_code, hourlyRate, isManager (default false), isActive (default true), manager (Association to Employees)
- **Settings** : `managed` – key ID (= 1), ALERT_THRESHOLDS ('75,90,100'), AT_RISK_PERCENT (80), TAX_PERCENT (18), PAYMENT_DAYS (30)

`db/schema.cds`, part 2: `context transaction`, every entity with the `managed` aspect.

Entity	Fields	Relations
Projects	PROJECT_CODE, PROJECT_NAME, DESCRIPTION, BILLING_TYPE, STATUS, PRIORITY, START_DATE, END_DATE, PLANNED_HOURS, CONSUMED_HOURS	CLIENT, MANAGER (Employees); `BUDGET : Composition of one Budgets`; `MILESTONES : Composition of many`; ALERTS, INVOICES, TIME_ENTRIES (Association to many)
Budgets (1 : 1)	BUDGET_AMOUNT, BILLING_RATE, RETAINER_FEE, CURRENCY_CODE, CONSUMED_AMOUNT, REMAINING_AMOUNT, CONSUMED_PERCENT, BUDGET_STATUS	PROJECT
Milestones	MILESTONE_NAME, DESCRIPTION, START_DATE, END_DATE, COMPLETION_PERCENT, STATUS	PROJECT; `TASKS : Composition of many`
Tasks	TASK_NAME, DESCRIPTION, START_DATE, END_DATE, PRIORITY, STATUS, ESTIMATED_HOURS, ACTUAL_HOURS, BURN_PERCENT	MILESTONE, ASSIGNEE (Employees)
Timesheets (header)	WEEK_START, WEEK_END, TOTAL_HOURS, BILLABLE_HOURS, STATUS, SUBMITTED_DATE, APPROVED_DATE, REJECTION_REASON	EMPLOYEE, APPROVED_BY; `Entries : Composition of many TimesheetEntries on Entries.PARENT = $self`
TimesheetEntries (items)	WORK_DATE, HOURS Decimal(4,2), IS_BILLABLE, NOTES	PARENT, TASK, INVOICE
Invoices : `common.Amount`	INVOICE_NO, PERIOD_FROM, PERIOD_TO, INVOICE_DATE, DUE_DATE, PAYMENT_DATE, BILLABLE_HOURS, STATUS (Draft/Issued/Paid/Cancelled)	PROJECT; Entries (Association to many on INVOICE)
Alerts	THRESHOLD, CONSUMED_PERCENT, MESSAGE	PROJECT, TASK
AuditLog	TIMESTAMP, USER_ID, ACTION, ENTITY, ENTITY_KEY, DETAILS	–

Entity	Fields	Relations
view **CDSViews.ProjectTimeEntries** (`db/CDSView.cds`)	entry + task name + employee name + timesheet status + invoice no	PROJECT (via TASK.MILESTONE.PROJECT)

Simplifications compared with the FRD:

- Manager is merged into Employees.
- Department is left out.
- The invoice gets the client through its project.
- Status code lists are CDS enums with `@assert.range`.

Seed data (CSV)

- Clients: 3 clients.
- Employees: 5 employees:
  - Meera Iyer – manager/admin, `meera.iyer@psa-demo.com`
  - Arjun
  - Rahul Sharma – user, `rahul.sharma@psa-demo.com`
  - Priya
  - Kiran – inactive
- Projects: 3 projects (T&M `PRJ-2026-001`, Fixed `PRJ-2026-002`, Retainer `PRJ-2026-003`) with budgets.
- Milestones and tasks: 4 milestones and 6 tasks (one *Done*, one unassigned).
- Timesheets: 4 timesheets (Approved, Submitted × 2, Draft) with entries.
- Invoices: 1 paid invoice.

---

6. Service layer (`srv/service.cds`, `srv/service.js` in CommonJS, `srv/test.http`)

There is one service, `@requires: 'authenticated-user' service PSAService` at `/odata/v4/psa`.

Projection	Draft	Authorization	Extra (calculated) columns
ClientSrv	yes	Admin `*`	–
EmployeeSrv	yes	User READ `where email = $user`; Admin `*`	`fullName`
ProjectSrv	yes	Admin `*`	`HOURS_PERCENT`; virtual `BURN_RATE`, `PROJECTED_END` (filled in `after READ`); `TIME_ENTRIES` redirected to ProjectTimeSrv; action `recalculateBudget()`
BudgetSrv	(composition)	Admin `*`	`CRITICALITY` (Breached 1, At Risk 2, else 3)
MilestoneSrv	(composition)	Admin `*`	`STATUS_CRITICALITY`
TaskSrv	(composition)	User READ; Admin `*`	`PROJECT_CODE`, `CRITICALITY` (burn ≥ 100 → 1, ≥ 80 → 2, else 3)
TimesheetSrv	yes	User READ/WRITE/submitTimesheet `where EMPLOYEE.email = $user`; Admin `*`	`STATUS_CRITICALITY`, `IS_LOCKED` (true unless Draft/Rejected); actions `submitTimesheet()`, `importHours(rows)`
TimesheetEntrySrv	(composition)	User `where PARENT.EMPLOYEE.email = $user`; Admin `*`	–

Projection	Draft	Authorization	Extra (calculated) columns
ApprovalSrv	no, `@readonly`, `@cds.redirection.target: false`	Admin	projection on Timesheets `where STATUS <> 'Draft'` (decided timesheets stay visible with their status), Entries redirected to TimesheetEntrySrv; actions `approveTimesheet()`, `rejectTimesheet(reason)`
InvoiceSrv	no, `@readonly`	Admin	`PAYMENT_STATUS` (Overdue when issued and past due), `STATUS_CRITICALITY`; actions `issueInvoice()`, `markInvoicePaid(paymentDate)`, `cancelInvoice()`
ProjectTimeSrv	no, `@readonly`, `@cds.redirection.target: false`	Admin READ	hours booked on a project (Timesheets facet)
SettingsSrv	yes	Admin READ/UPDATE (no create/delete)	–
AuditLogSrv	no, `@readonly`	Admin READ	–
AlertSrv	no, `@readonly`	Admin READ	`CRITICALITY`

Operations:

- `generateInvoice(in : many $self, projectID, periodFrom, periodTo) returns InvoiceSrv` – Admin, bound to the InvoiceSrv **collection** so Fiori reloads the list after it (an unbound action does not refresh the table)
- unbound `function whoAmI() returns WhoAmI` – all users
- `function getBurnSummary(projectID) returns BurnSummary` – Admin: budget, consumed, remaining, %, status, planned / consumed hours, burn rate (approved hours per week since the start), projected completion (today + remaining hours / burn rate)

Read-only **analytics projections** (Admin, `@cds.redirection.target: false`) for the KPI cards and the chart. Each one has a counter or amount column that Fiori sums with `$apply`:

Projection	Based on	Measures
`ProjectAnalytics`	Projects + Budget + Client (flattened)	`BUDGET_AMOUNT`, `CONSUMED_AMOUNT`, `REMAINING_AMOUNT`, `PLANNED_HOURS`, `CONSUMED_HOURS`, `ACTIVE_COUNT` (END_DATE ≥ `current_date`), `AT_RISK_COUNT`
`TimesheetAnalytics`	Timesheets	`PENDING_COUNT` (status Submitted)
`InvoiceAnalytics`	Invoices	`DRAFT_COUNT`, `OVERDUE_AMOUNT` (Issued and past due)
`EntryAnalytics`	TimesheetEntries	`UNBILLED_HOURS` (approved, billable, not invoiced)

Annotations in the service:

- Mark system-filled fields `@readonly` (consumed and remaining amounts, burn and completion %, timesheet status, totals, approver).

Actions:

- Report success with `req.notify(...)`, so that Fiori shows a toast (`req.info` opens a dialog).
- Reject with clear messages and proper codes (400, 403, 409).

---

7. Fiori UI (FRD §9) – make it look good and follow SAP Fiori standards

Each app keeps its annotations in `app/<app>/annotations.cds` (starting with `using PSAService as service from '../../srv/service';`), and `app/services.cds` collects them with `using from './<app>/annotations';`. Field labels come from `db/i18n`; the app files only add labels for calculated fields or where a screen needs another text.

General rules

- Every entity gets `UI.HeaderInfo` with `TypeImageUrl` (an SAP icon), titles and descriptions.
- Lists use `UI.LineItem` with `@UI.Importance: #High` on key columns, plus `UI.SelectionFields`, `UI.PresentationVariant` sorting, value helps and labels.
- Technical keys are `@UI.Hidden`.
- Status columns use `Criticality` + `CriticalityRepresentation: #WithIcon`.
- Percent fields use `@Measures.Unit: '%'`, so progress bars read "12 %" and not "12.00 of 100".
- `UI.DataPoint` with `Visualization: #Progress` is used for budget burn, hours used, milestone completion and task burn.
- `UI.HeaderFacets` on object pages, `UI.CollectionFacet` for grouped sections.
- Each action gets:
  - `Core.OperationAvailable` (e.g. Submit only when `IsActiveEntity` and status is Draft/Rejected; Issue only for Draft; Mark as Paid only for Issued), and
  - `Common.SideEffects` so the page refreshes.
- `UI.UpdateHidden` / `UI.DeleteHidden: IS_LOCKED` on timesheets.

Apps. Each app is a folder `app/<name>` like the SAP Fiori generator creates it in the sample project: `annotations.cds`, `package.json`, `ui5.yaml`, `eslint.config.mjs`, `README.md` and `webapp/` with manifest.json, Component.js, index.html and `i18n/i18n.properties`. SAPUI5 **1.153.0** from `https://sapui5.hana.ondemand.com/1.153.0/`, theme `sap_horizon`. Manifest: `"i18n"` and `"@i18n"` models, `"title": "{{appTitle}}"`, `crossNavigation.inbounds` with the app's intent, `flexEnabled: true`. List Report apps use the **flexible column layout** (`rootView` `sap.fe.templates.RootContainer.view.Fcl`, `routerClass` `sap.f.routing.Router`, route targets as arrays, `controlAggregation` `beginColumnPages` / `midColumnPages` / `endColumnPages`, default `TwoColumnsMidExpanded`).

Tile / App	Template	Key elements
Manager Home (`home`, start page of the Admin)	**Analytical List Page** (List Report with `views` primary = `UI.PresentationVariant` → chart, secondary = `UI.LineItem`, Grid Table) on `ProjectAnalytics`	**KPI cards** in the header (manifest `keyPerformanceIndicators` → `UI.KPI` annotations): *Active Projects*, *At-Risk Budgets*, *Pending Approvals* (from `TimesheetAnalytics`); each opens a detail chart. **Chart** *Budget vs. Consumed per project* (`UI.Chart`, measures = `Aggregation.CustomAggregate` with `@Aggregation.default: #SUM`). **Table** with filters (client, billing type, budget status), column widths via `@HTML5.CssDefaults`; row → outbound navigation to `Project-manage`, which opens the object page because `ProjectSrv` has `@Common.SemanticKey: [PROJECT_CODE]`. **Buttons** (`UI.DataFieldForIntentBasedNavigation`) to Approvals, Billing Workbench, My Timesheets, Clients, Employees.
My Timesheets (`timesheets`, User + Admin)	List Report + Object Page	Header: Total Hours, Billable Hours, Status. *Hours* table with inline creation (date, task value help, hours, billable checkbox, notes, task burn % in colour). *Week* section with submitted / approved dates. Buttons *Create/Save*, *Import Hours*, *Submit for Approval*.
Approve Timesheets (`approvals`, Admin)	List Report + Object Page	Filters for status (default *Submitted*), employee and week. *Approve* / *Reject* (reason dialog) both above the table (multi-select) and in the object page header.

Tile / App	Template	Key elements
Projects (`projects`, Admin)	List Report + Object Page + sub object page (Milestone)	Header with project info, *Budget Used* progress, *Hours Used* progress, Budget Status, Remaining Budget. Header field group: client, **status**, manager, billing type. Facets: Overview (General incl. description/status/priority, Schedule incl. burn rate and projected completion, Budget), Milestones & Tasks (milestone status in colour), **Timesheets** (`TIME_ENTRIES`), Scope-Creep Alerts, Invoices, Change Log (created/changed by/at). *Recalculate Budget* action.
Billing Workbench (`invoices`, Admin)	List Report + Object Page	**KPI cards** *Unbilled Hours* (`EntryAnalytics`), *Draft Invoices*, *Overdue Amount* (`InvoiceAnalytics`). *Generate Invoice* (project, period), *Issue*, *Mark as Paid* (payment date), *Cancel*. Status column shows *Overdue*. Header: total, status, due date.
Clients, **Employees** (Admin)	List Report + Object Page	Master data with field groups and communication annotations.
Settings (`settings`, Admin)	List Report + Object Page (draft, `Capabilities.InsertRestrictions/DeleteRestrictions` false)	alert thresholds, at-risk %, tax %, payment days
Audit Log (`auditlog`, Admin)	List Report + Object Page (read-only)	time, user, action, object, details; filters user / action / object / time

Launchpad: `app/index.html`, **FLP sandbox** `test-resources/sap/ushell/bootstrap/sandbox.js`

1. Before the FLP starts, make a synchronous `GET /odata/v4/psa/whoAmI()` call, with a safe fallback if it fails.
2. Build `applications` only for the user's role. Each entry uses `applicationType: "URL"`, `additionalInformation: "SAPUI5.Component=psa.<app>"` and `url: "./<app>/webapp"`. Intents: `Manager-home`, `Timesheet-manage`, `Approval-manage`, `Project-manage`, `Invoice-manage`, `Client-manage`, `Employee-manage`, `Settings-manage`, `AuditLog-display`. Give every app its **own semantic object**: Fiori Elements shows an intent button only if the launchpad returns exactly one app for it.
3. Start page per role: `renderers.fiori2.componentData.config.rootIntent` = `Manager-home` for an Admin, `Timesheet-manage` for a User.
4. **Important:** do **not** define your own `LaunchPage.adapter.config.groups`. The CDN sandbox loads its own `fioriSandboxConfig.json` ("Sample Applications") and merges it over yours. In `sap.ui.getCore().attachInit`, keep only the group `sap_ushell_generated_group_id` and rename it.
5. Set the real user in `services.Container.adapter.config`.
6. Call `sap.ushell.Container.registerLogout(() => location.href = "/do/logout")`.
7. Disable search, notifications and personalization.

---

8. Security and deployment files

xs-security.json

- `xsappname: psa-application`.
- Scopes `$XSAPPNAME.Admin` and `$XSAPPNAME.User`, role templates Admin and User, role collections `PSA_Admin` and `PSA_User`.
- Redirect URIs: `https://*.applicationstudio.cloud.sap/**`, `https://*.hana.ondemand.com/**`, `http://localhost:5000/**`.

package.json (CommonJS, no `"type": "module"`), `engines.node >= 22`.

- Dependencies: `@sap/cds ^10`, `@cap-js/hana`, `@sap/xssec`.
- devDependencies: `@sap/cds-dk`, `@cap-js/sqlite`, `@cap-js/cds-test`, `mbt`, `rimraf`.
- `cds.requires`:
  - `db: hana`, `auth: xsuaa` **always** (not only in a `[production]` profile: the deployed server must never fall back to SQLite / mocked users),
  - `[test]`: in-memory SQLite and mocked users `meera.iyer@psa-demo.com` / `admin` (Admin) and `rahul.sharma@psa-demo.com` / `user` (User), all default CAP mock users disabled.
- `sapux`: the app folders.
- Scripts (sample-project style):
  - `start: cds-serve`, `watch: cds watch --profile test --open index.html`, `watch-<app>` per app
  - `test: CDS_ENV=test node --test test/psa.test.js`
  - `build: rimraf resources mta_archives && mbt build --mtar archive`, `deploy: cf deploy mta_archives/archive.mtar --retries 1`, `undeploy: cf undeploy psa-application --delete-services --delete-service-keys --delete-service-brokers`

Approuter: directly in `app/` (sample-project style)

- `app/package.json` (name `app`, `@sap/approuter`, `start`), `app/package-lock.json`.
- `app/xs-app.json`: welcome file `/index.html`, logout endpoint `/do/logout`, `sessionTimeout`; routes with `localDir` for `^/index.html$` and `^/<app>/webapp/(.*)$` (the approuter serves the UI files), then `^/(.*)$` → destination `srv-api`; all `authenticationType: xsuaa`, CSRF on.
- `app/default-env.json`: `srv-api` → `http://localhost:4004`, for hybrid runs.

mta.yaml, ID `psa-application`:

- before-all: `npm ci`, `npx cds build --production`.
- Modules:
  - `psa-application` (approuter, nodejs, path `app`, builder `npm-ci`, 256 MB, ignore `node_modules/` and `default-env.json`)
  - `psa-application-srv` (nodejs, `gen/srv`, 512 MB)
  - `psa-application-db-deployer` (hdb, `gen/db`, 256 MB)
- Resources:
  - `psa-application-db` (hdi-shared)
  - `psa-application-auth` (xsuaa application, `xsappname: psa-application-${org}-${space}`, role collections `PSA_Admin (${org}-${space})` and `PSA_User (${org}-${space})`, redirect URI of the approuter)

---

9. Run and deploy instructions (put these in README.md)

Hybrid in BAS (Full Stack Cloud Application dev space, HANA Cloud running and mapped to the space):

```
git clone https://github.com/saurabh-1907/Saurabh-Pandey-Sprint-Evaluation && cd Saurabh-Pandey-Sprint-Evaluation
npm install
cf login
cf create-service xsuaa application psa-hybrid-auth -c xs-security.json
cds bind auth --to psa-hybrid-auth
cds build --production
cds deploy --to hana
cds watch --profile hybrid
# second terminal, from the PROJECT ROOT (the binding is in .cdsrc-private.json):
npm install --prefix app
cds bind --exec --profile hybrid -- npm start --prefix app
```

Open port **5000** to reach the XSUAA login and then the launchpad. Assign `PSA_Admin` / `PSA_User` in *Security* → *Users*. BAS port URLs work only in a browser that is logged in to BAS.

Cloud Foundry (public URL)

```
npm run build    # mbt build -> mta_archives/archive.mtar
npm run deploy   # cf deploy
cf apps          # URL of "psa-application" = the public link
```

Then, in the cockpit, go to *Security → Role Collections* and add users to `PSA_Admin (org-space)` / `PSA_User (org-space)` with the **Default identity provider**. The role collections are tied to the XSUAA instance, so the hybrid `PSA_Admin` does not work for the deployed app. Every user needs an Employee record with the same e-mail. On a trial account HANA stops nightly: start it again in HANA Cloud Central.

---

10. Tests (`test/psa.test.js`, `node --test` + `@cap-js/cds-test`)

Write automated tests (≥ 38) that cover:

- **Roles and visibility**
  - A User sees only their own timesheets and own employee record.
  - A User gets 403 on projects, clients and invoices.
- **KPI analytics**: `$apply=aggregate(...)` on the analytics projections returns the KPI numbers; a User gets 403 on them.
- **Project draft creation** with budget and validation errors (duplicate code, budget ≤ 0, dates).
- **User timesheet draft**
  - defaults (employee, Monday, entry defaults) and Monday adjustment
  - duplicate week
  - every entry rule (0.25–24 h, inside week, ≤ 24 h/day, duplicate task + day, Done task)
- **Submit, lock and delete**
  - Submit works only for Draft/Rejected.
  - A submitted timesheet is locked: draftEdit returns 409 and DELETE returns 409.
- **Approval rules**
  - no self-approval (403)
  - reject needs a reason (400)
  - a rejected timesheet can be re-submitted
- **Roll-up after approval**
  - task, milestone, project and budget values
  - alerts raised once per threshold
- **Invoices**
  - T&M: 78 h × 100 = 7,800 + 18 % tax 1,404 = 9,204
  - Fixed: capped at the budget
  - Retainer: flat 8,000, and a duplicate period is rejected
  - Linked entries make the timesheet *Invoiced*, and nothing is billed twice.
- **Invoice lifecycle**: Draft → Issued → (Overdue) → Paid with payment date; future payment date rejected; Cancel releases the hours; invalid transitions are rejected.
- **Project insight**: milestone status, `getBurnSummary`, burn rate / projected end / `TIME_ENTRIES` on the project.
- **Import hours**: valid row imported, duplicate ignored, unknown task / outside week / Done task reported; locked timesheet rejected.
- **Settings and audit log**: Admin changes tax via draft; User gets 403; the audit log contains create, submit, approve, reject, import and invoice entries.

---

11. Known pitfalls (avoid them)

HANA and the service layer

- **HANA + CASE:** `@cap-js/hana` turns the *simple* CASE form (`case STATUS when 'X' then …`) into invalid SQL (`when 'X' = true`). Always use the *searched* CASE form: `case when STATUS = 'X' then 1 … end`.
- **Redirections:** a second projection on the same entity needs `@cds.redirection.target: false` or an explicit `redirected to … on …`.

Fiori and UI5

- **UI5 version:** use 1.153.0 from the SAPUI5 CDN for every app and the launchpad, and give the standalone index.html a full-height style. Otherwise the page is blank.
- **CDN sandbox merge:** see §7. Never filter the FLP to a group id you defined yourself.
- **KPI cards:** the KPI value must be an `Aggregation.CustomAggregate` (with `@Aggregation.default`), and a currency/unit of an amount KPI needs one too (`#MAX`), because Fiori aggregates it as well. Put `UI.KPI.DataPoint` inline, not as a path.
- **Toasts:** use `req.notify` for success messages (toast). `req.info` opens a dialog.

Login and roles

- **Approuter in hybrid mode:** start it from the project root with `--prefix`. Otherwise it fails with "no UAA service found".
- **Missing roles:** if the token has only `openid`, the role collection is not assigned for the identity provider the user logged in with. Assign it to the `sap.default` user, then log out and log in again.

---

12. Definition of done

☐ `npm test` passes.

☐ `cds build --production`, `cds deploy --to hana` and `cds watch --profile hybrid` work.

☐ `npm run build` + `npm run deploy` work, and the public URL leads to the SAP login and then the launchpad.

☐ Admin starts on the Manager Home (ALP), User starts on My Timesheets. No *Sample Applications*, no Forbidden pages, no hand-written views or CSS.

☐ Manager Home shows the KPI cards *Active Projects, At-Risk Budgets, Pending Approvals*, the budget chart and table with drill-down; Billing Workbench shows *Unbilled Hours, Draft Invoices, Overdue Amount*.

☐ End-to-end flow works in the browser:

1. The user creates a week, adds hours and submits. A toast appears and the timesheet is locked.
2. The admin approves. Task, milestone, project and budget update, and alerts appear.
3. The admin generates an invoice, issues it and marks it as paid.

☐ README explains the problem → solution, the data model, roles, apps, flow, BAS steps, CF steps and tests.